Brickts SQL course — Week 9 — Keep APIs and queries fast

sparql-playground / lessons/sql

Generated for offline / Kindle reading

Week 9 — Keep APIs and queries fast

This pack concatenates the six daily lessons and the weekly challenge, then the Week primary-reading links from SOURCES.md. Work the SQL in `psql` against `brickts_course`; keep notes under ignored `student-work/sql/`.

Contents

- Day 57: Day 57 — Read an execution plan before adding an index
- Day 58: Day 58 — Page history with a stable cursor

- Day 59: Day 59 — Write a parameterized history and plot contract
- Day 60: Day 60 — Bound database concurrency and prove cancellation
- Day 61: Day 61 — Make ingest retries idempotent
- Day 62: Day 62 — Budget a bounded history workload
- Day 63 (weekly challenge): Day 63 — Independent challenge — a fast, bounded history API design

---

Day 57 — Read an execution plan before adding an index

Previous: Day 56 · **Week 9 · 45–75 minutes.** Prerequisites: Days 1–56, or equivalent SQL skills.

Goal

Explain how a bounded historian query reaches its rows and decide whether an index serves that access pattern.

Before you start

Run the lab check. Keep the 24,190-row fixture intact. Use scratch copies in `student` for index changes and `ANALYZE`; record server version, query parameters and cache conditions. An observed plan on this fixture is evidence about this fixture.

Concept

The grain is one site, opaque point UUID and instant. The sample primary key supports equality on site and point followed by a time range. `EXPLAIN` estimates work; `EXPLAIN (ANALYZE, BUFFERS)` executes the statement and reports observed work. Read estimated versus actual rows, loops, filters, sorting and buffers together. A sequential scan can be sensible for a small table or broad range. A `LIMIT` bounds returned rows but does not guarantee a bounded scan or aggregate.

Tiny example

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT s.ts, s.value
FROM course.sample AS s
WHERE s.site_id = 'SYN-NORTH'
  AND s.point_id = (
    SELECT p.point_id FROM course.point AS
         p
    JOIN course.equipment AS e USING
         (site_id, equipment_id)
    WHERE p.site_id = 'SYN-NORTH'
      AND e.label = 'Supply B' AND p.role
         = 'kw')
  AND s.ts >= '2026-01-05
         10:00Z'::timestamptz
  AND s.ts < '2026-01-05
         11:00Z'::timestamptz
ORDER BY s.ts LIMIT 12;
```

This demonstration returns twelve six-kW samples. The metadata subquery resolves identity before history selection.

SQL challenge

Make a scratch sample copy and compare a scoped, one-hour north Supply A kW query with and without an appropriate composite index. Use January 5 at 15:00 UTC: twelve source rows include eleven eligible values. Preserve bad/null rows in raw history, then

express eligibility separately. Explain why a value-only index is a poor starting point for this read contract.

Experiment

Widen the range to both fixture days. Predict how selectivity changes before capturing both plans. Repeat each query three times; report individual timings without promising a speedup.

Acceptance checks

- Results agree before and after indexing, including quality.
- Your plan annotation identifies estimated rows, actual rows, loops and sorting.
- You distinguish returned-row limits from work limits.

Stretch and reflection

Compare a covering index with the primary key. What extra writes and storage would its additional columns require?

Save your SQL, actual output, and one explanation in `student-work/sql/day57/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 56 ·

Day 58 — Page history with a stable cursor

Previous: Day 57 · **Week 9 · 45–75 minutes.** Prerequisites: Days 1–57, or equivalent SQL skills.

Goal

Design keyset pagination that preserves ties, site scope and a fixed history window.

Before you start

Retrieve point UUIDs through metadata. Keep a fixed half-open window in every page. Work in SQL or a request-contract document; no application UI is needed.

Concept

For one point, `(site_id, point_id, ts)` makes timestamp unique. Across points, timestamp needs a UUID tie breaker. A cursor must carry the ordering tuple and bind to the original site, selected points, range and model revision. Treat it as opaque validated input at an API boundary. Keyset pagination avoids

walking a growing offset, but concurrent inserts before the cursor can still change what a multi-request reader sees. Declare snapshot/export semantics or document that behavior.

Tiny example

```
WITH events(ts, event_id) AS (
  VALUES ('2026-01-05
         08:00Z'::timestamptz, 1),
         ('2026-01-05
         08:00Z'::timestamptz, 2),
         ('2026-01-05
         08:05Z'::timestamptz, 3)
)
SELECT * FROM events
WHERE (ts, event_id) > ('2026-01-05
         08:00Z'::timestamptz, 1)
ORDER BY ts, event_id LIMIT 2;
```

Both remaining events appear. Filtering only `ts > cursor_time` would lose event 2.

SQL challenge

Write a prepared query that pages north Supply A duct static history over January 5, 08:00–09:00 UTC, with page size four. Its resolved point has ten rows because two instants are absent. Return a next cursor only when another row exists; fetching at most

page size plus one can establish this. Then describe the ordering needed for a two-point endpoint.

Experiment

In a scratch copy, add an earlier row between pages and a later row inside the window. Record which each pagination contract sees. Compare offset and keyset behavior; do not label either a complete snapshot without evidence.

Acceptance checks

- Concatenated pages contain ten distinct source rows in increasing order.
- Changing site, point set or window invalidates a cursor.
- Empty and final pages terminate without repeating the last row.
- A declared maximum page size and maximum time range constrain requests independently.

Stretch and reflection

How would a data revision or immutable export identifier make a long download reproducible?

Save your SQL, actual output, and one explanation in `student-work/sql/day58/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 57 ·

Day 59 — Write a parameterized history and plot contract

Previous: Day 58 · **Week 9 · 45–75 minutes.** Prerequisites: Days 1–58, or equivalent SQL skills.

Goal

Specify a safe SQL boundary and a small plot-ready response whose scope and units remain visible.

Before you start

Use the model-to-point resolution learned in week 8. Review the fixture contract. Write SQL and example JSON as local artifacts; this lesson does not add an endpoint to brickts.

Concept

A query contract states accepted types, bounds, ordering, authorization scope and failure behavior. Parameter binding supplies values, including UUIDs and timestamps.

Identifiers such as table names and sort directions require a fixed allowlist; quoting an arbitrary input does not grant authorization. A SQL `PREPARE` demonstrates binding without choosing a web framework. A plot client needs timestamps, values, units, quality or gaps, and provenance. It does not need the whole joined metadata table for every sample.

Tiny example

```
PREPARE day59(text, uuid, timestamptz,
         timestamptz, integer) AS
SELECT ts, value, quality
FROM course.sample
WHERE site_id = $1 AND point_id = $2
  AND ts >= $3 AND ts < $4
ORDER BY ts LIMIT $5;
```

Validate `$3 < $4`, allowed window, authorized point ownership and a positive capped `$5` before execution. A limit does not replace these checks. Client-driver placeholders can differ from PostgreSQL prepared-statement syntax.

SQL challenge

Define a one-point six-hour history contract and a fifteen-minute aggregate contract. Include good sample count and expected count in each aggregate bucket so missing

data cannot masquerade as zero. Resolve north Supply A kW for January 5, 15:00–16:00 UTC: raw history has twelve rows, eleven eligible values, and a visible gap at 15:10. Make a minimal JSON example with units, half-open range, identity, model revision, query version and row count.

Experiment

List outcomes for an invalid UUID, reversed interval, excessive limit, unknown point, wrong site, and an input containing a quote. Distinguish validation errors from valid empty history. Execute the prepared SQL with valid fixture parameters.

Acceptance checks

- Query text contains no interpolated user values.
- The response represents null/quality without inventing measured zero.
- Scope applies to metadata resolution and sample retrieval.
- Plot arrays cannot silently mismatch timestamps and values.

Stretch and reflection

Which provenance fields belong once per response, and which belong on individual buckets?

Save your SQL, actual output, and one explanation in `student-work/sql/day59/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 58 ·

Day 60 — Bound database concurrency and prove cancellation

Previous: Day 59 · **Week 9 · 45–75 minutes.** Prerequisites: Days 1–59, or equivalent SQL skills.

Goal

Show that a request deadline actually cancels database work and leaves its connection reusable.

Before you start

Use only your dedicated lab database and at most two sessions. Choose a five-second maximum experiment and identify each session's PID yourself. Never cancel a PID copied from somebody else's activity listing.

Concept

A connection pool limits concurrent sessions; a request queue also needs a bound and a wait deadline. Otherwise overload merely moves

into the queue. A client giving up does not by itself prove that its SQL stopped. PostgreSQL `statement_timeout` cancels server work. A canceled statement in an explicit transaction leaves the transaction aborted until rollback. Pools must clean up failed transactions before handing a connection to another request. A useful record separates queue wait, query time and serialization time.

Tiny example

Run interactively so you can recover after the intentional error. In psql, set `\set VERBOSITY verbose` first to display SQLSTATE:

```
BEGIN;
SET LOCAL statement_timeout = '200ms';
SELECT pg_catalog.pg_sleep(2);
-- Expected SQLSTATE 57014; issue the
         following after the error:
ROLLBACK;
SELECT 1 AS connection_reusable;
```

The delay is bounded even if the timeout setting fails. `SET LOCAL` lasts only for its transaction.

SQL challenge

Specify a two-connection pool, a bounded waiting queue and distinct acquisition/query deadlines for the day59 contract. Demonstrate a timeout, rollback and successful next query. In a second session, observe your first session with `pg_stat_activity`; capture PID, state, wait event and query age without logging credentials or private query text. Explain how the driver propagates cancellation when a caller disconnects.

Experiment

Compare the server timeout above with interrupting a two-second delay from the client. Observe whether the backend stops work and whether the transaction requires rollback. If you cannot inspect activity, mark that evidence unavailable rather than declaring cancellation verified.

Acceptance checks

- SQLSTATE and elapsed time document actual server cancellation.
- The same session successfully executes a query after cleanup.

- Pool capacity, queue capacity and both deadlines are explicit.
- Your activity observation distinguishes waiting from executing.

Stretch and reflection

When would `lock_timeout` expose a blocked query sooner than the statement deadline? Keep all settings local to this lab.

Save your SQL, actual output, and one explanation in `student-work/sql/day60/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 59 ·

Day 61 — Make ingest retries idempotent

Previous: Day 60 · **Week 9 · 45–75 minutes.** Prerequisites: Days 1–60, or equivalent SQL skills.

Goal

Define a retry policy that survives duplicate batches and partial failure without hiding contradictory measurements.

Before you start

Use a new scratch table in `student`. Do not modify the fixture or stop a database container. Simulate errors with transactions and a disconnected lab client.

Concept

The logical sample key is `(site_id, point_id, ts)`. A unique constraint makes duplicates detectable. `ON CONFLICT DO NOTHING` can make identical retries harmless, but also silently retain an old value when a later batch disagrees. A correction policy must declare

whether updates are permitted, which source wins and how changes are audited. Batch identity and sample identity answer different questions. A transaction commits an entire declared batch or none of it; losing the connection after commit leaves the client uncertain and requires safe replay.

Tiny example

```
CREATE TABLE student.retry_demo (
  site_id text NOT NULL, point_id uuid NOT
        NULL,
  ts timestamptz NOT NULL, value double
        precision,
  PRIMARY KEY (site_id, point_id, ts)
);
INSERT INTO student.retry_demo
SELECT site_id, point_id, ts, value FROM
        course.sample
ORDER BY site_id, point_id, ts LIMIT 3
ON CONFLICT DO NOTHING;
```

Run the insert again. The count remains three. This only proves duplicate-key behavior for this chosen policy.

SQL challenge

Design a scratch ingest ledger and a transaction for a twelve-row, single-point batch. A repeated identical batch must create no new logical samples; a changed value for

an existing key must be rejected or recorded as an explicit revision. Include source quality and finite-value validation. Demonstrate rollback when the last row violates a constraint, then replay the valid batch.

Experiment

Disconnect a client after a small commit, reconnect and replay the same batch. Separately attempt a connection to an unused local port with a short connection timeout to represent unavailability. Record bounded retries and backoff in a design note. Do not claim to have reproduced every network failure mode.

Acceptance checks

- Two identical submissions produce twelve logical rows.
- One invalid row prevents partial batch visibility.
- A conflicting value has an observable, declared outcome.
- The outage path stops after a bounded retry budget and preserves the batch for later replay.

Stretch and reflection

Which evidence distinguishes “not committed” from “commit acknowledgment lost”?

Save your SQL, actual output, and one explanation in `student-work/sql/day61/`. Use only `brickts_course`; see this week’s primary references.

Previous: Day 60 ·

Day 62 — Budget a bounded history workload

Previous: Day 61 · **Week 9 · 45–75 minutes.** Prerequisites: Days 1–61, or equivalent SQL skills.

Goal

Turn query limits into a measurable workload budget across raw history, aggregates and response serialization.

Before you start

Keep the test at two clients, forty requests total and the two-day fixture. Set a stop condition of ten seconds per request and a small total deadline. Use SQL transcripts or an optional local driver; no HTTP server is required.

Concept

A history request has several costs: identity resolution, connection wait, SQL execution, result transfer and encoding. Selecting fewer columns reduces transfer, while aggregation

must still scan eligible input rows unless a suitable summary exists. Limits on points, window, page size and concurrency describe different bounds. For a one-point day at five-minute cadence, the expected maximum is 288 source rows. Unknown coverage and a valid empty result need distinct reporting.

Tiny example

```
SELECT pg_catalog.pg_column_size(ROW(ts,
         value, quality)) AS tuple_bytes
FROM course.sample
WHERE site_id = 'SYN-NORTH'
  AND ts >= '2026-01-05
         00:00Z'::timestamptz
  AND ts < '2026-01-06
         00:00Z'::timestamptz
ORDER BY point_id, ts LIMIT 3;
```

This measures PostgreSQL's representation of a sample row, not JSON bytes, protocol overhead or total process memory. Use actual encoded payload size when you make a claim about the wire.

SQL challenge

Prepare three bounded query shapes: a one-point raw hour, a one-point day, and one equipment's hourly kW summary. Include time predicates before aggregation, explicit quality policy and deterministic ordering.

Record request parameters, rows returned, eligible source count, execution time and encoded response bytes where measured. Compare one client with two; state pool and queue settings from day60.

Experiment

Repeat a warm query and then change its point/window. Explain why the second request does not necessarily share the same cache conditions. Compare a three-column payload with a redundant joined history payload for the same rows. Do not fetch the entire database merely to discard rows in a client.

Acceptance checks

- Every query is scoped by site and a finite time range.
- Raw one-point day responses never exceed 288 fixture rows.
- No missing sample is converted into measured zero.
- A compact table contains actual counts and timings, plus unavailable measurements labeled as such.
- The run stops at the declared bounds even after errors.

Stretch and reflection

Which budget would need to change before adding twenty points to one plot? Justify each change with data.

Save your SQL, actual output, and one explanation in `student-work/sql/day62/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 61 ·

Day 63 — Independent challenge — a fast, bounded history API design

Previous: Day 62 · **Week 9 review · 2–4 hours, split across sessions as needed.** Prerequisites: Days 57–62, or equivalent skills.

Project brief

Produce a reviewable SQL and API design for a historian read path. A caller selects an authorized site and RDF point, requests a finite interval, and receives raw or aggregated plot data. Reuse the fixture and your earlier experiments. Implement SQL and contract artifacts; an application UI or new running service is unnecessary.

This is an independent assessment. Choose your own decomposition and use earlier lessons and primary references as references after outlining your approach.

Acceptance criteria

- Identity resolves through model metadata to opaque UUIDs; site authorization covers both resolution and SQL.
- Parameterized query contracts cap time span, points, rows, queue and execution time.
- Pagination has deterministic ordering and declared consistency semantics.
- Plots retain units, quality, gaps, query/model versions and coverage.
- Measured plans support index choices, and cancellation evidence includes transaction cleanup.
- Duplicate ingest and database unavailability have bounded, observable outcomes.

Deliverables

Submit SQL, sample request/response documents, one plan comparison, a small workload table and an error matrix. Include empty, wrong-site, duplicate and timeout cases. Keep tests within two clients and forty requests. Label results **observed**, **fixture-only** or **not run**.

Self-review

What work can still grow despite a response limit? Which measurements support your claimed latency? What happens when a client abandons a request? Explain one limitation that would require a new experiment before production use.

Previous: Day 62 ·

Primary reading — Week 9

Use the lesson first, then these references for precise behavior. Full list: SOURCES.md.

Week 9

- Using EXPLAIN
- Multicolumn indexes
- LIMIT/OFFSET
- Resource settings
- Statement and lock timeouts